AI-Powered Intelligent Hiring Assistant

Project Analysis Report

---

1. Executive Summary

This report documents the structure, technology stack, and functionality of the AI Resume Matcher project as submitted in the reviewed archive. The system is a resume-to-job-description matching tool that combines three complementary scoring signals — semantic similarity, keyword (TF-IDF) similarity, and rule-based skill matching — into a single explainable match score, delivered through both a Streamlit web application and a FastAPI REST backend, with an added conversational assistant for candidate follow-up questions.

The archive contains 8 files: 6 Python modules, one dependency manifest, and one pre-existing project description document (project_summary.docx). No automated test suite, deployment configuration, or example dataset was found in the archive.

2. Archive Contents

File	Type	Size	Role
app.py	Python	~410 lines	Streamlit web application (frontend)
api.py	Python	~349 lines	FastAPI REST backend
resume_matcher.py	Python	~273 lines	Core ML matching engine (ResumeJobMatcher class)
chatbot.py	Python	~416 lines	Retrieval-based conversational assistant (ResumeChatbot class)
config.py	Python	~256 lines	Centralized configuration (Config / Dev / Prod / Testing)
download_nltk_data.py	Python	~66 lines	One-time NLTK corpus downloader utility
requirements.txt	Text	37 lines	Python dependency manifest
resume_matcher_model.pkl	Binary	~5 KB	Serialized TF-IDF vectorizer + skills list
project_summary.docx	Word	~31 KB	Pre-existing project description document

3. System Architecture

The project follows a layered architecture with a shared ML core consumed by two independent front ends:

- resume_matcher.py — shared AI/ML engine, imported by both interfaces below
- app.py — Streamlit UI, calls the engine directly in-process
- api.py — FastAPI REST service, wraps the same engine for programmatic / external access
- chatbot.py — conversational layer, imported by both app.py and api.py, answers questions grounded in the engine's own analysis output
- config.py — shared settings (file limits, scoring thresholds, environment profiles)

Neither front end persists analysis history to a database; results are held in memory (Streamlit session state, or an in-process dictionary keyed by session_id in the API).

4. AI / ML Approach

The core scoring logic lives in ResumeJobMatcher.calculate_similarity_scores() and combines three signals into one weighted overall score:

Signal	Technique	Library	Weight
Semantic similarity	Sentence embeddings + cosine similarity	sentence-transformers (all-MiniLM-L6-v2)	40%
Keyword similarity	TF-IDF vectorization + cosine similarity	scikit-learn TfidfVectorizer	30%
Skill match	Substring match against a fixed skill list	Custom Python logic	30%

Text preprocessing (lowercasing, punctuation/digit removal, tokenization, stopword removal, lemmatization) is applied before the semantic and TF-IDF comparisons via NLTK (punkt, stopwords, wordnet corpora).

Match categories

The combined score is mapped to one of four labels:

- ≥ 0.80 — Excellent Match
- ≥ 0.60 — Good Match
- ≥ 0.40 — Fair Match
- < 0.40 — Poor Match

Skills database — discrepancy noted

project_summary.docx states the system recognizes "250+ technical skills across various domains," but the actual skills_keywords list in resume_matcher.py currently contains 19 hard-coded terms (e.g. python, java, react, sql, aws, docker, kubernetes). This is worth reconciling before submitting the description document alongside the code — either by expanding the skills list or updating the description to match the current implementation.

5. Conversational Assistant (chatbot.py)

ResumeChatbot is a retrieval-based intent classifier: candidate messages are matched against a bank of example phrasings for ~20 intents using TF-IDF cosine similarity, combined with a keyword-overlap bonus so short or blunt questions (e.g. "score?") still classify confidently. This is a template-retrieval design — answers are selected from fixed, pre-written text, not generated freely by a language model.

Intents requiring a prior analysis result

- ask_score, ask_category — overall score and match category
- ask_matched_skills, ask_missing_skills — skill overlap detail
- ask_semantic_score, ask_tfidf_score, ask_skill_score — per-signal breakdown
- ask_recommendations — improvement suggestions

General intents (no analysis required)

- greeting, thanks, goodbye, ask_capabilities
- ask_explain_scoring, ask_supported_files
- ask_resume_length, ask_ats_tips, ask_keyword_tips
- ask_summary_tips, ask_format_tips, ask_soft_skills_tips, ask_cover_letter

Both app.py and api.py maintain the assistant's grounding context per session: app.py stores the last analysis in Streamlit session state, and api.py keeps an in-memory dict keyed by a session_id passed alongside each /analyze and /chat call.

6. REST API (api.py)

Built with FastAPI. Endpoints exposed:

Method	Path	Purpose
GET	/	Health check / root
GET	/health	Health check
POST	/analyze	Analyze resume text vs. job description
POST	/analyze/upload	Analyze an uploaded resume file (PDF/DOCX) vs. job description
POST	/batch-analyze	Analyze multiple resumes against one job description
GET	/skills	Return the configured skills database
POST	/extract-skills	Extract recognized skills from arbitrary text
POST	/similarity	Compute similarity scores between two arbitrary texts
POST	/chat	Conversational assistant, grounded via session_id

Custom 404 and 500 exception handlers are registered. CORS middleware is enabled for cross-origin access.

7. Web Application (app.py)

Built with Streamlit. Main sections, in page order:

- Sidebar — usage instructions and loaded-model information
- Resume Input — paste text, or upload a PDF/DOCX (extracted via PyPDF2 / python-docx)
- Job Description input
- Analysis Results — overall score and category
- Skills Analysis — matched vs. missing skills
- Recommendations — generated improvement suggestions
- Detailed Scores — semantic / TF-IDF / skill breakdown, visualized with Plotly gauge and score charts
- Export Results — downloadable analysis report
- 💬 Ask the AI Assistant — chat interface backed by ResumeChatbot

8. Configuration & Utilities

config.py

Defines a base Config class plus DevelopmentConfig, ProductionConfig, and TestingConfig subclasses, selected via a get_config(env) factory function. Centralizes settings such as file upload limits and scoring parameters rather than hard-coding them across app.py / api.py.

download_nltk_data.py

Standalone utility that downloads the three NLTK corpora the matching engine depends on (punkt, stopwords, wordnet). resume_matcher.py also calls the equivalent download step itself on startup, so this script is primarily useful for pre-provisioning an environment (e.g. inside a Docker build step) ahead of time.

resume_matcher_model.pkl

A pickled dictionary containing the fitted TF-IDF vectorizer, the skills keyword list, and the sentence-transformer model name (the embedding model itself is reloaded by name via SentenceTransformer(), not pickled directly).

9. Key Dependencies (requirements.txt)

Category	Packages
NLP / ML	sentence-transformers, scikit-learn, nltk, transformers, torch
Data	pandas, numpy
Web (UI)	streamlit, plotly
Web (API)	fastapi, uvicorn, python-multipart

Category	Packages
File parsing	PyPDF2, python-docx
Utilities	joblib, requests, python-dotenv, wheel, setuptools

10. Strengths

- Multi-signal scoring (semantic + keyword + skills) is more robust than any single method alone, and each component is independently visible to the user — the design is explainable, not a black box.
- Clear separation of concerns: one shared ML engine consumed by two independent interfaces (Streamlit and FastAPI).
- The chatbot's hybrid TF-IDF + keyword-bonus classification handles short/blunt questions well without needing an external API.
- Graceful degradation: file parsing, model loading, and chat classification all have explicit fallback/error paths rather than crashing outright.

11. Limitations & Recommendations

- Skills database is small (19 terms) relative to what project_summary.docx claims (250+) — reconcile before submission.
- The chatbot answers are retrieved from a fixed template bank, not generated freely — phrasing for context-specific answers (score, skills) is always identical across users. A retrieval-augmented generation (RAG) layer with a language model could produce more natural, varied answers grounded in the same analysis data, while keeping the current system as an always-available fallback.
- No automated tests (unit or integration) were found in the archive.
- No persistent storage — analysis history and chat context live only in memory and are lost on restart.
- project_summary.docx references Docker, Kubernetes, and CI/CD deployment configs that are not present in this archive — the description document appears to describe a broader target state than what is currently implemented.

12. Conclusion

The AI Resume Matcher is a functionally complete resume-screening tool with a sound multi-signal scoring design, dual (UI + API) access, and a working rule-based conversational assistant layered on top. The main gaps relative to its own description document are the size of the skills database and the absence of the deployment tooling mentioned there — both are straightforward to address in a future iteration without changing the core architecture.